

Nordvik Industrial Group

A multi-company ERP data warehouse, with machine-learning anomaly detection

SOURCE ROWS	DBT TESTS	DATA TRAPS	AVG PRECISION
396,055	70	10	0.831

This report documents the design and measured results of a dimensional data warehouse consolidating six subsidiaries running four incompatible ERP systems, and a machine-learning monitoring layer built to catch the two failure classes the warehouse's own test suite structurally cannot detect.

The finding worth reading for: machine learning is the right answer for one of the two defect classes examined and the *wrong* answer for the other. Both comparisons are published, including the one where a single line of SQL beats the model outright.

Author	Sagar Kumar Mishra
Repository	github.com/sagarkumarmishra/industrial-group-edw
Generated	2026-09-18
Stack	dbt 1.12 · DuckDB · Snowflake-portable SQL · scikit-learn · pandas
Reproducible	make clean && make all

The data is synthetic and deliberately adversarial. Nothing in this report has run in production; the engineering is real, the deployment is not claimed. Limitations are stated in section 9 rather than omitted.

1. The problem

Six factories, acquired over twenty years, each still running the system it came with. Every month the finance team asks one question:

“What did we actually spend on stock last month?”

Answering it means combining six extracts that disagree about almost everything: how a date is stored, what unit a duration is in, whether a missing value is blank or 1970-01-01, even whether the same warehouse code refers to the same warehouse. Do it by hand and you get a number. It is simply not the right number, and nothing tells you that.

Why silent failure is the real problem

A pipeline that crashes is a good day — somebody notices in minutes. The expensive failures are the silent ones: the job succeeds, the report renders, the totals look like totals.

One subsidiary books labour in **seconds**, with the unit recorded in a separate column that is easy to overlook. Sum the hours column without checking it and group labour comes out **192 times too high** for the affected companies. Nothing errors. No test fails.

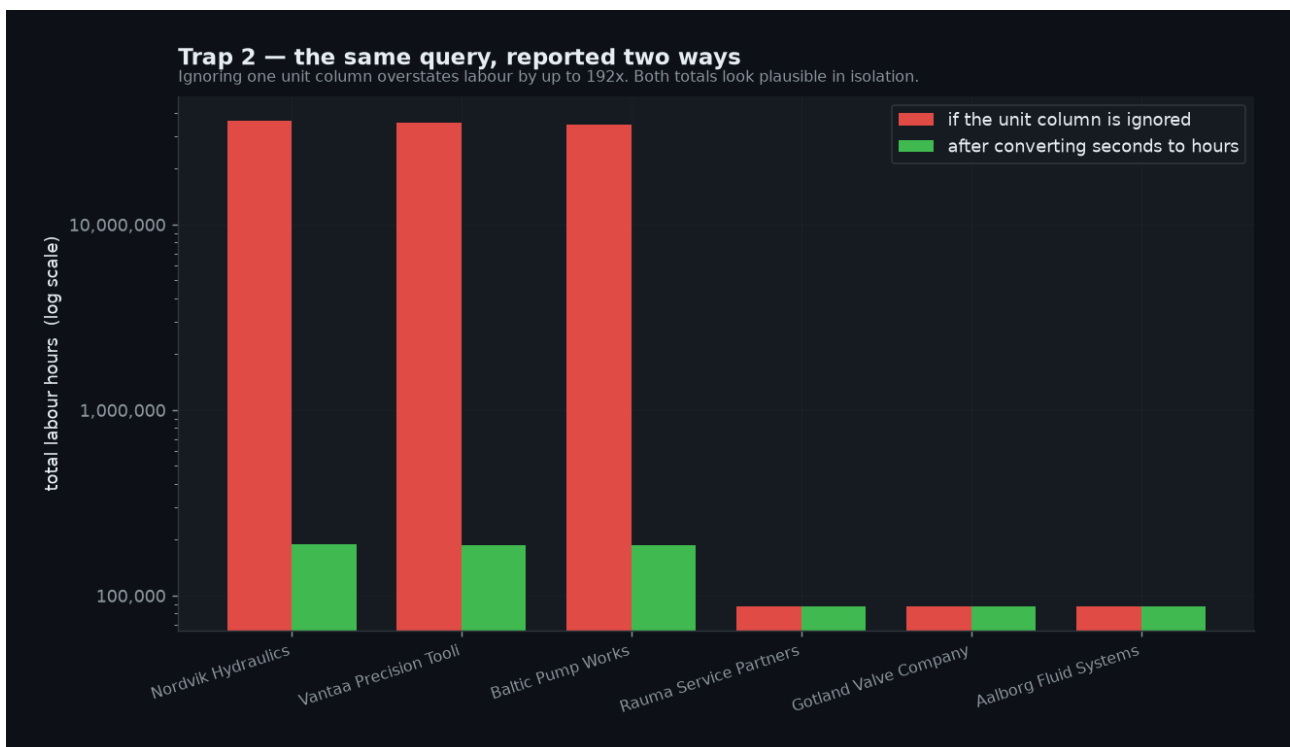


Figure 1. The same query, reported two ways. The three subsidiaries on the modern ERP book in seconds; the legacy three do not, which is why only half the group is affected and why the error survives a spot check.

Nine further traps of this character are implemented and documented. Each is drawn from production multi-company ERP work, and each shares the same property: the data is wrong in a way that looks right.

2. The ten data traps

The generator injects these deliberately. Clean synthetic data proves nothing — anyone can model tidy inputs.

#	Trap	What it does if missed
1	Mixed time zones	Modern ERP stores UTC, legacy stores local. Converting both, or neither, corrupts every date key.
2	Mixed units in one column	Hours and seconds in the same column. Overstates labour 192x.
3	Sub-day grain	line_seq does not restart daily. A date-based key silently collapses distinct transactions.
4	Corrupt quantities	Absurd quantities. Cost derived as qty x rate explodes; the source's own amount is correct.
5	Epoch as null	1970-01-01 means "no date". Left alone it joins cleanly and reports 53 years early.
6	Fixed-width padded text	Untrimmed codes make one value look like two, splitting every aggregate.
7	Reserved words	A source table named ENTITY. Unquoted DDL fails on it.
8	Column drift	Not every subsidiary sends the same columns.
9	Genuine source duplicates	Nine rows really were extracted twice. Deduplicating hides an upstream fault.
10	Contextual price anomalies	Prices 2-9x the item's own median, every value individually plausible. No rule can express this.

Trap 10 was added last, deliberately, after the warehouse was complete. The other nine are all deterministically detectable, so a model applied to them would be theatre. **After injecting 162 contextual price anomalies, all 70 existing tests still passed** — which is the evidence that the blind spot is real rather than manufactured.

3. Architecture

Five layers, each with one responsibility. A defect is fixed in exactly one place, and nothing downstream repeats the work.

Layer	Responsibility	Objects
raw	Land extracts untouched. Padding, epoch dates and reserved words all preserved, so what the source said remains provable.	10 sources
staging	Resolve all ten traps, once each, with the reason recorded beside the code that implements it.	10 models
intermediate	Union three incompatible time-entry sources onto one grain, so nothing downstream needs to know which system a booking came from.	1 model
marts	Dimensional model. Conformed dimensions, surrogate keys, unknown members at -1, degenerate dimensions retained.	6 dims, 4 facts
ml	Monitoring. Contextual anomaly detection and volume/freshness scoring over the marts, not over raw.	4 scripts

Built objects

Fact table	Rows
fct_inventory_transaction_cost	200,079
fct_labour_hours	150,000
fct_work_order_lead_time	25,000
fct_labour_hours_variance	16,380

Dimension	Rows
dim_company	7
dim_item	901
dim_warehouse	31
dim_cost_component	5
dim_date	1,826
dim_service_module	5

The decision that mattered most

The grain of the inventory fact includes the full timestamp at second precision, not a truncated date. Transaction timestamps are unique to the second and line_seq does **not** restart daily, so a date-based key collapses genuinely distinct transactions onto one row and the total quietly drops.

A date-based key looks correct when you spot check it, because roughly 80% of rows carry line_seq = 1. That is what makes it dangerous. A uniqueness test on the full grain now guards it.

4. Testing

70 automated tests run on every build: uniqueness, not-null, referential integrity, accepted values, accepted ranges, and six singular tests asserting business plausibility rather than structure.

The current build is **PASS=90, WARN=1, ERROR=0** across 91 nodes. The single warning is intentional and must not be “fixed”.

A test configured to warn, not to pass

Nine rows are genuine duplicates in one subsidiary's extract — trap 9. Deduplicating them would hide a real upstream fault and make the fact disagree with source on row count. So the known baseline is encoded in the test itself:

```
warn_if: >0    # surface it every run, so nobody forgets
error_if: >9    # fail the build the moment it gets worse
```

An assertion of zero would have been quietly disabled by the third person who hit it. An assertion of “no worse than the nine we have documented” stays meaningful and is a real regression guard.

5. Machine learning: scope and justification

The ML layer is scoped to the two failures the test suite cannot reach, and the comparison against SQL rules is published in both directions.

Defect class	Nature	Verdict
CORRUPT_QUANTITY	Absolute — wrong on sight	Rule wins decisively. Do not deploy a model.
CONTEXTUAL_PRICE	Relative — wrong only for that item	Model wins. No threshold can express it.

Why one is easy

A quantity of 256,000,000 on an inventory line is nonsense regardless of context. One line of SQL catches every instance, exactly, with no training and no drift.

Why the other is not

```
item A-1042 (a washer)    unit_cost = 41.80    usual: 4.60
item B-7781 (a pump)      unit_cost = 41.80    usual: 44.20
```

Identical value. The first is a nine-fold error; the second is a Tuesday. No global threshold separates them, because the catalogue spans parts costing 2 and parts costing 900. The only way to judge the row is against that item's own history, and maintaining 900 per-item thresholds by hand is not a plan.

Features: mostly a story about exclusions

Two mart columns encode the answer. `is_quantity_reliable` is the rule output; `amount_if_recomputed` becomes astronomic exactly when quantity is corrupt. Include either and the model scores near-perfectly having learned nothing — you would ship it, the dashboard would look excellent, and the first novel defect would sail straight through.

This is the most common way ML pipelines produce impressive, worthless numbers. Both columns are excluded, and `build_features()` raises an `AssertionError` if either reappears. A comment would be ignored by whoever adds a column to improve a score next year.

Of the eleven features retained, the load-bearing ones are contextual: a robust z-score of price within the item, and the ratio of the row's price to that item's median. Median and MAD rather than mean and standard deviation, because the anomalies are *inside* the series being summarised — a mean is dragged upward by the very rows being hunted, so each anomaly partly conceals itself.

Model choice

Isolation Forest, 300 estimators, unsupervised. Unsupervised is the point: in production nobody has told you which rows are wrong. The ground truth here exists only to score the result afterwards — the model never sees it, which is what makes the approach transferable rather than a demonstration that works only because the answers happened to be available.

A supervised classifier would score better here and be useless in production, since it needs the labels you are trying to produce. Local Outlier Factor was comparable and far slower at 200k rows; an autoencoder is defensible at much larger scale but a heavier dependency for results within noise of the forest.

6. Results

ROWS SCORED	AVG PRECISION	TOP-20 PRECISION	VOLUME FLAGS
200,070	0.831	1.000	5/5,400

Corrupt quantity — 57 rows in ground truth

Detector	Precision	Recall	F1
SQL rule quantity > 1e6	1.000	1.000	1.000
Isolation Forest	0.097	0.684	0.170

The rule is perfect. The forest is far worse. **The correct engineering decision is to keep the rule and not deploy a model for this class** — adding one to a solved problem costs money and credibility.

Contextual price — 162 rows in ground truth

Detector	Precision	Recall	F1
SQL rule, unit_cost global p99.9	0.537	0.667	0.595
Isolation Forest, contamination 0.002	0.362	0.895	0.515
Isolation Forest — average precision	—	—	0.831



Figure 2. The reversal between the two panels is the finding. Left: a one-line WHERE clause scores 1.000 on both precision and recall. Right: no threshold works, and the model is the only option.

The uncomfortable result, reported rather than buried

At the default threshold the forest's F1 of 0.515 is **worse** than the crude rule's 0.595. That is a real result and it stays in the output.

But read it correctly. Average precision is 0.831, so the *ranking* is excellent. The problem is not the model, it is where the line was drawn. **Strong ordering, weak decision** — different failures with different fixes, and conflating them is how teams discard models that were working.

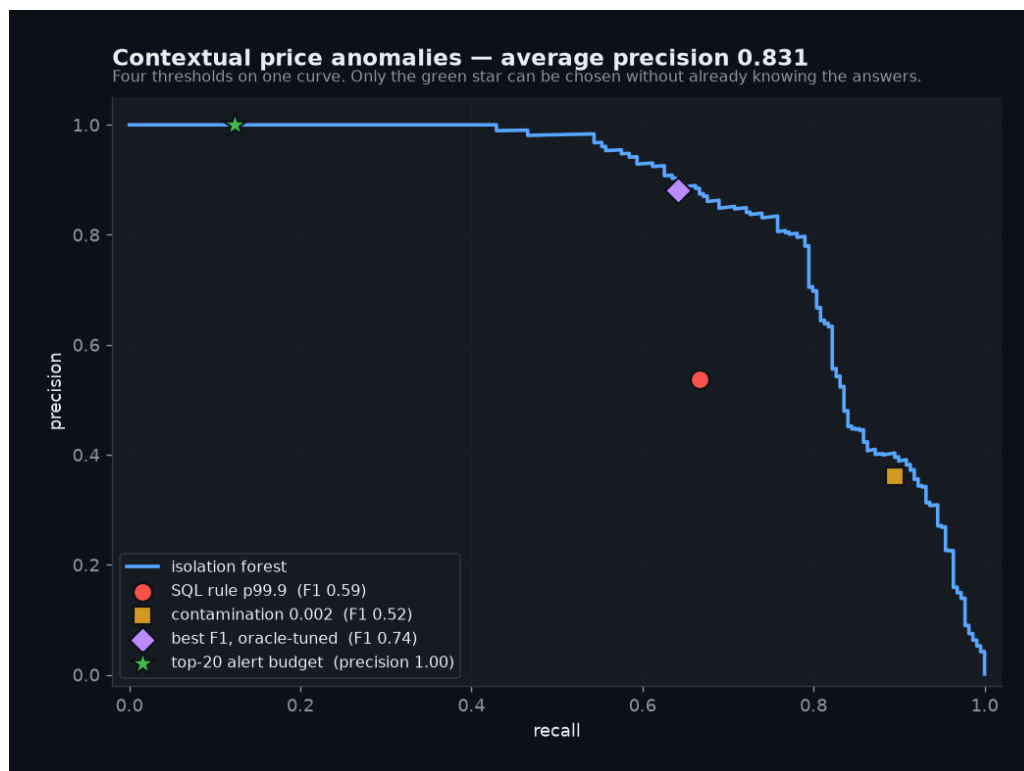


Figure 3. The red dot is the SQL rule, sitting clearly below the curve: at matched recall the model is strictly more precise. The rule only wins on F1 because the chosen contamination put the forest at a poor point on a curve it otherwise dominates.

7. Choosing a threshold

Average precision measures how well a model *orders* rows. It says nothing about where to cut, and the cut is what determines whether anyone acts on the output. This is the step most write-ups skip.

Operating point	Precision	Recall	F1	Usable in production?
Contamination 0.002	0.362	0.895	0.515	Yes, but arbitrary
Best F1, oracle-tuned	0.881	0.642	0.743	No — uses the labels
Top-20 alert budget	1.000	0.123	0.220	Yes. This ships.

The oracle-tuned row is an upper bound and nothing more. If you have the labels you do not need the detector. It is reported because knowing the ceiling is useful, and labelled at every appearance because quoting it as achievable performance would be dishonest.

The alert budget needs no labels at all. A human can meaningfully triage roughly 20 rows a day, so take the top 20 by score. **All 20 were genuine anomalies — precision 1.000, zero false alarms.**

Recall of 0.123 sounds poor until you compare it to the alternative, which is a report nobody reads because two-thirds of it is noise. An alert stream that is always right earns the trust that lets you widen the budget later.

One that cries wolf gets muted in a week, and then recall is zero.

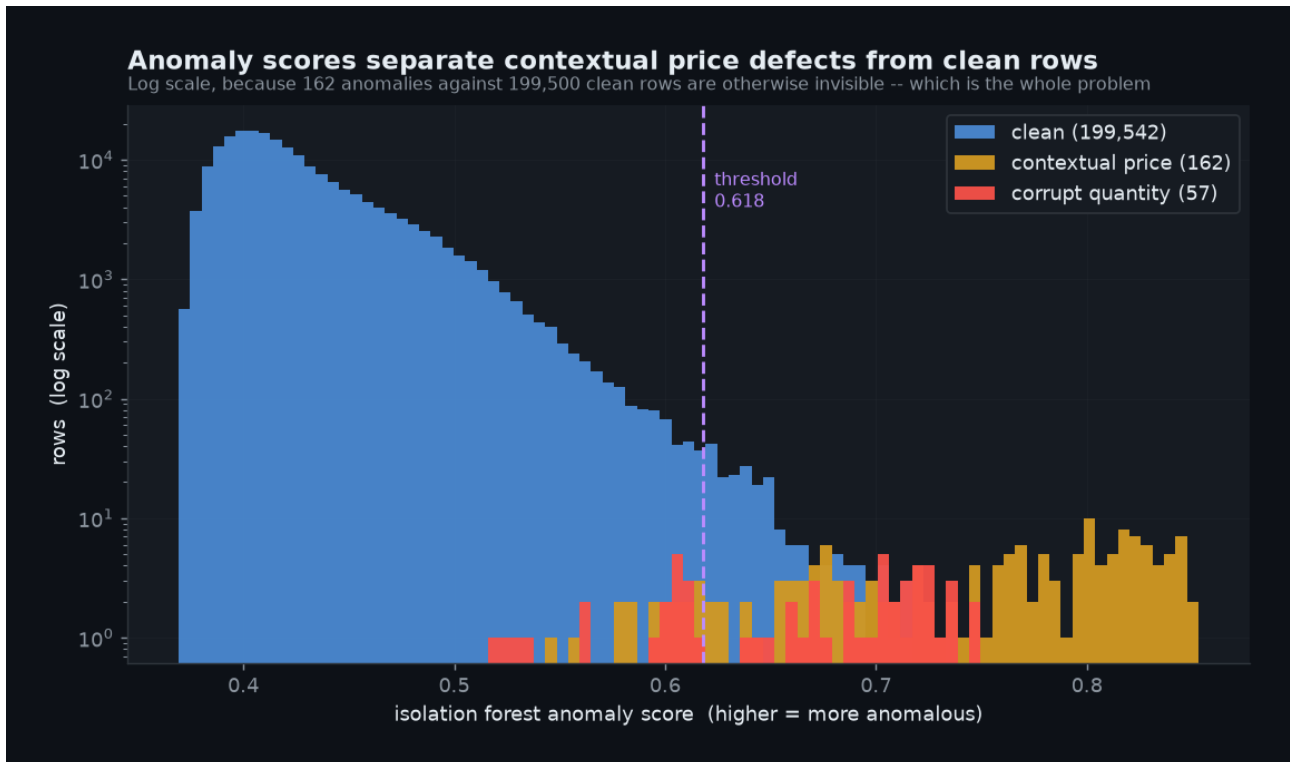


Figure 4. Log scale, because 162 anomalies against 199,500 clean rows are otherwise invisible — which is the whole problem. Note the genuine overlap: clean rows reach 2.34x the item median while anomalies start at 2.0x, so this is a real tradeoff rather than a threshold anyone could eyeball.

8. The blind spot no row-level test can cover

All 70 tests inspect rows that **exist**. If a subsidiary's extract fails for three days: no test fails, no key is null, every foreign key resolves, and the fact table reconciles *perfectly* to the source — because the source is short too. Three days of one company's stock movements are simply gone.

This is the most common incident class in production data platforms, and the one most projects have no answer to at all.

The detail that makes or breaks it

A naive implementation groups by date and company and looks for low counts. **It finds nothing**. A day with zero rows produces no group — the absence is invisible to a GROUP BY. The fix is to reindex onto a complete date x company spine and fill the gaps with explicit zeros. That single step is the difference between a monitor that works and one that reports all-clear through a total outage.

Why a Poisson residual, not a z-score

Daily row counts are *count* data, roughly Poisson: variance grows with the mean, so the spread is already determined by the expected value. A robust z-score on ~36 rows a day divides by a MAD of about 4, so an ordinary quiet Tuesday of 23 rows scores -4.7 and pages somebody. The Poisson residual divides by the square root of 36, scores it -2.3, and correctly ignores it — while a zero-row day still scores -6.1.

I arrived at that the way everyone does: the first version flagged twelve days, of which three were the real outage and the rest were Tuesdays.

Threshold	Poisson residual flags	Robust z-score flags
3.0	23	63
4.0	6	14
5.0	5	5
6.0	5	2

The two agree only at 5.0, and the Poisson residual is better on both sides. Loosen to 4.0 and the z-score flags 14 days against 6, the extras being quiet Tuesdays. Tighten to 6.0 and the z-score has **already lost three of the five genuine incidents**. This is not a marginal accuracy gain — it is a usable operating *range* instead of one lucky threshold.

Result

5 anomalous company-days out of 5,400 checked, every one genuine. 3 zero-row days from a three-day extract outage, and 2 days where another subsidiary's job ran twice. Precision 1.000, recall 1.000, no false positives.



Figure 5. Red band: three consecutive days at zero rows. Amber: six times normal volume. Every dbt test passes on all 5,400 of these company-days.

9. Ground truth, and a bug worth documenting

The generator decides which rows to break, so it knows the answers. They are written to an `ml` schema that `sources.yml` does not declare — so no dbt model can reference it even accidentally. The warehouse is built without ever seeing which rows are defective.

Defect type	Rows	Caught by
EPOCH_DATE	300	dbt test (deterministic)
CONTEXTUAL_PRICE	162	Isolation Forest
CORRUPT_QUANTITY	57	SQL rule
SOURCE_DUPLICATE	9	dbt test (deterministic)
Total	528	

The bug

My first label join used `txn_timestamp`, the obvious key. It silently dropped **46% of the labels**, and the model looked far worse than it was.

The cause: staging converts timestamps to the reporting timezone for the three subsidiaries whose systems store UTC. That is **trap 1** — the first trap this project documents — biting my own tooling. For half the group the raw timestamp no longer equalled the value in the fact table.

The fix was to key on business columns plus the measures, which pass through staging unchanged. The reasoning is now a comment in the generator rather than a silent fix, because a trap that catches the person who invented it is better evidence than any claim I could make about it being realistic.

10. Limitations

Stated rather than omitted, because overclaiming is worse than a modest scope.

- **The data is synthetic.** Deliberately adversarial and modelled on real patterns, but no real company's data is in this repository.
- **Nothing here has run in production.** No live traffic, no business rollout, no on-call rotation. The engineering is real; the deployment is not claimed.
- **One seeded run.** Reproducible via `make all`, but not a cross-validated study. No confidence intervals are claimed.
- **The oracle-tuned F1 is an upper bound, not a result.** It uses the labels to pick its threshold, which is impossible in production.
- **The centred rolling window is a backtest convenience.** A monitor scoring today cannot see the future and must use a trailing window, which performs slightly worse. Noted because this is exactly how backtests come to flatter live systems.
- **No drift handling.** Production would need periodic refitting and a check on whether the score distribution has moved.
- **No SCD2, no incremental models, no orchestration.** Full rebuilds are instant at 200k rows and unacceptable at 200m. The keys for incrementality exist and are unused.

11. What this demonstrates

Claim	Where to verify it
Dimensional modelling of awkward multi-source data	models/marts/, docs/architecture.md
Knowing which data problems are dangerous, and why	docs/data-traps.md
Testing for correctness rather than green ticks	models/marts/_marts.yml, tests/
ML scoped to where it earns its place	docs/ml-anomaly-detection.md
Reporting results honestly, including negative ones	ml/detect_anomalies.py output
Documenting decisions so others can maintain the work	docs/decision-log.md

Every figure and metric in this report was generated from a single run on 2026-09-18. Reproduce with `make clean && make all`.